Servlets

Servlets are **small Java programs** that run on the **server side** of a web application. They are used to handle requests from clients (browsers) and generate **dynamic web content**.

- **Servlets extend web server functionality**.
- They are part of **Java-based web development**.
- Servlets are mainly used to create **interactive and dynamic web pages**.

A servlet is a program that **receives a request, processes it, and sends back a response**.

How Web Browsers and Servers Work

Before understanding servlets, we must know how the web works.

Static Web Page Processing

1. User enters a URL in the browser.
2. Browser sends an **HTTP request** to the server.
3. Server finds the requested file (HTML page).
4. Server sends it back as an **HTTP response**.
5. Browser displays the page.

- Example: Simple HTML page (same content for all users)

Dynamic Web Page Processing

- In real applications (like online shopping), data changes frequently.
    - Example: Product price , Availability , Orders

So, pages must be **generated dynamically** based on database data.

CGI (Common Gateway Interface)

Before servlets, **CGI** was used for dynamic content.

How CGI Works

- For every request:
    - A **new process** is created.
    - It connects to a database.
    - Sends response back to user.

Disadvantages of CGI (Drawbacks)

- Very **slow performance**
- High **memory and CPU usage**
- Not **platform independent**
- Opens and closes database connections repeatedly

Due to these problems, **Servlets were introduced**

Advantages of Servlets over CGI

Servlets solved all major issues of CGI:

1. Better Performance

- Runs inside server memory
- No need to create separate process for each request

SBC Karkala

2. Platform Independent

- Written in Java → works on any system

3. Secure

- Uses Java security model to protect server resources

4. Reusability & Integration

- Can use Java libraries
- Can interact with:
  - Databases
  - Applets
  - Other systems

Conclusion: Servlets are **faster, secure, and efficient** than CGI

Servlet Life Cycle

The life cycle of a servlet consists of three main methods: **init(), service(), and destroy()**. These methods are implemented by every servlet and are invoked at specific times by the server.

When a user enters a URL in a web browser, the browser generates an HTTP request and sends it to the server. The server receives the request, maps it to a particular servlet, and loads the servlet into its memory.

1. **init() Method**

   The server calls the init() method only once when the servlet is first loaded into memory. This method is used to initialize the servlet. Initialization parameters can also be passed so that the servlet can configure itself.

2. **service() Method**

   After initialization, the server calls the service() method to process the HTTP request. The servlet reads the data provided in the request and generates an appropriate HTTP response. This method is called for every request, and the servlet remains in memory to handle multiple requests.

3. **destroy() Method**

   Finally, when the server decides to remove the servlet from memory, it calls the destroy() method. This method is used to release resources such as file handles and to save important data. After this, the memory used by the servlet can be garbage collected.

Steps to Develop and Run a Servlet

- Create a servlet class
- Compile the program
- Configure it in web.xml
- Run using web server
- Access through browser

Servlet is configured in web.xml by defining servlet name and URL mapping.

SBC Karkala

A Simple Servlet

A simple servlet is a Java program that handles client requests and generates a response.

- A servlet class is created by extending GenericServlet.
- The service() method is overridden to process requests.
- It uses ServletRequest to read data and ServletResponse to send output.
- setContentType() sets the response type.
- **getWriter()** method returns a **PrintWriter object**.
  It is used to **send response data from the servlet to the client browser**.
- The servlet is compiled and configured in web.xml.
- It is executed using a web server and accessed through a browser.

Basic Servlet Program to Display Welcome Message

```
import java.io.*;
import javax.servlet.*;
public class WelcomeServlet extends GenericServlet {
   public void service(ServletRequest request, ServletResponse response)
          throws ServletException, IOException {
      response.setContentType("text/html");
      PrintWriter pw = response.getWriter();
      pw.println("<html><body>");
      pw.println("<h2>Welcome to Servlet Programming!</h2>");
      pw.println("</body></html>");
      pw.close();
   }
}
```

Explanation

- The servlet class **extends GenericServlet**.
- The **service() method** is overridden to handle client requests.
- setContentType("text/html") sets the response type.
- getWriter() is used to send output to the browser.
- The message **"Welcome to Servlet Programming!"** is displayed.

Servlet API

The Servlet API consists of two packages: **javax.servlet** and **javax.servlet.http**. These packages contain the classes and interfaces required to build servlets.

The javax.servlet Package

The **javax.servlet** package contains several interfaces and classes that provide the framework in which servlets operate.

SBC Karkala

It includes the core components required for developing servlets. The most important interface in this package is the **Servlet** interface. All servlets must either implement this interface or extend a class that implements it.

The **ServletRequest** and **ServletResponse** interfaces are also very important. These interfaces are used for handling client requests and generating responses.

Interface	Description
Servlet	Declares life cycle methods for a servlet.
ServletConfig	Allows servlets to get initialization parameters.
ServletContext	Enables servlets to log events and access information about their environment.
ServletRequest	Used to read data from a client request.
ServletResponse	Used to write data to a client response.

The following table summarizes the core classes that are provided in the **javax.servlet** package:

Class	Description
GenericServlet	Implements the **Servlet** and **ServletConfig** interfaces.
ServletInputStream	Provides an input stream for reading requests from a client.
ServletOutputStream	Provides an output stream for writing responses to a client.
ServletException	Indicates a servlet error occurred.
UnavailableException	Indicates a servlet is unavailable.

The Servlet Interface

All servlets must implement the **Servlet interface**. This interface defines the core methods that are required for the life cycle of a servlet.

The Servlet interface declares the methods **init()**, **service()**, and **destroy()**, which are called by the server at different stages of the servlet life cycle.

- The **init()**, **service()**, and **destroy()** methods are known as life cycle methods and are invoked by the server.
- The **getServletConfig()** method is used by the servlet to obtain initialization parameters.
- The **getServletInfo()** is used provide useful information such as author, version, and date.

Thus, the Servlet interface provides the basic structure required for creating and managing servlets.

Method	Description
void destroy( )	Called when the servlet is unloaded.
ServletConfig getServletConfig( )	Returns a **ServletConfig** object that contains any initialization parameters.
String getServletInfo( )	Returns a string describing the servlet.
void init(ServletConfig *sc*) throws ServletException	Called when the servlet is initialized. Initialization parameters for the servlet can be obtained from *sc*. An **UnavailableException** should be thrown if the servlet cannot be initialized.
void service(ServletRequest *req*, ServletResponse *res*) throws ServletException, IOException	Called to process a request from a client. The request from the client can be read from *req*. The response to the client can be written to *res*. An exception is generated if a servlet or IO problem occurs.

TABLE 31-1 The Methods Defined by **Servlet**

The ServletConfig Interface

The **ServletConfig** interface allows a servlet to obtain configuration data when it is loaded. The methods declared by this interface are summarized here:

Method	Description
ServletContext getServletContext( )	Returns the context for this servlet.
String getInitParameter(String *param*)	Returns the value of the initialization parameter named *param*.
Enumeration getInitParameterNames( )	Returns an enumeration of all initialization parameter names.
String getServletName( )	Returns the name of the invoking servlet.

The ServletContext Interface

The **ServletContext** interface enables servlets to obtain information about their environment. Several of its methods are summarized in Table 31-2.

Method	Description
Object getAttribute(String *attr*)	Returns the value of the server attribute named *attr*.
String getMimeType(String *file*)	Returns the MIME type of *file*.
String getRealPath(String *vpath*)	Returns the real path that corresponds to the virtual path *vpath*.
String getServerInfo( )	Returns information about the server.
void log(String *s*)	Writes *s* to the servlet log.
void log(String *s*, Throwable *e*)	Writes *s* and the stack trace for *e* to the servlet log.
void setAttribute(String *attr*, Object *val*)	Sets the attribute specified by *attr* to the value passed in *val*.

TABLE 31-2 Various Methods Defined by **ServletContext**

ServletRequest Interface

The ServletRequest interface allows a servlet to **get information from the client request**.

It provides methods to read request data.

Method	Description
Object getAttribute(String *attr*)	Returns the value of the attribute named *attr*.
String getCharacterEncoding( )	Returns the character encoding of the request.
int getContentLength( )	Returns the size of the request. The value –1 is returned if the size is unavailable.
String getContentType( )	Returns the type of the request. A **null** value is returned if the type cannot be determined.
ServletInputStream getInputStream( ) throws IOException	Returns a **ServletInputStream** that can be used to read binary data from the request. An **IllegalStateException** is thrown if **getReader( )** has already been invoked for this request.
String getParameter(String *pname*)	Returns the value of the parameter named *pname*.
Enumeration getParameterNames( )	Returns an enumeration of the parameter names for this request.
String[ ] getParameterValues(String *name*)	Returns an array containing values associated with the parameter specified by *name*.
String getProtocol( )	Returns a description of the protocol.
BufferedReader getReader( ) throws IOException	Returns a buffered reader that can be used to read text from the request. An **IllegalStateException** is thrown if **getInputStream( )** has already been invoked for this request.
String getRemoteAddr( )	Returns the string equivalent of the client IP address.
String getRemoteHost( )	Returns the string equivalent of the client host name.
String getScheme( )	Returns the transmission scheme of the URL used for the request (for example, "http", "ftp").
String getServerName( )	Returns the name of the server.
int getServerPort( )	Returns the port number.

TABLE 31-3 Various Methods Defined by **ServletRequest**

ServletResponse Interface

The ServletResponse interface allows a servlet to **send response to the client**.

It provides methods to generate output.

Method	Description
String getCharacterEncoding( )	Returns the character encoding for the response.
ServletOutputStream getOutputStream( ) throws IOException	Returns a **ServletOutputStream** that can be used to write binary data to the response. An **IllegalStateException** is thrown if **getWriter( )** has already been invoked for this request.
PrintWriter getWriter( ) throws IOException	Returns a **PrintWriter** that can be used to write character data to the response. An **IllegalStateException** is thrown if **getOutputStream( )** has already been invoked for this request.
void setContentLength(int *size*)	Sets the content length for the response to *size*.
void setContentType(String *type*)	Sets the content type for the response to *type*.

TABLE 31-4 Various Methods Defined by **ServletResponse**

The GenericServlet class

The GenericServlet class provides implementations of basic lifecycle methods of a servlet.

It implements the Servlet and ServletConfig interfaces.

Methods Provided by Default

- log(String s) – writes message to server log
- log(String s, Throwable e) – logs message with exception
- init() – initializes the servlet
- destroy() – cleans up resources

The ServletInputStream Class

The ServletInputStream Class extends InputStream.

It is used to read data from client request

The method readLine() to to read a line of data from the input stream.

int readLine(byte[] buffer, int offset, int size) throws IOException

- It reads bytes from the client request
- Stores them in the given *buffer* starting at *offset*
- Reads up to the specified *size*

It returns number of bytes read and -1 if end of stream is reached

The ServletOutputStream Class

The ServletOutputStream class extends OutputStream.

It is used to write data to a client response.

Servlet Exception Classes

The javax.servlet package defines two main exceptions:

1. ServletException

- Indicates that a **problem has occurred in the servlet**
- Used for general servlet errors

2. UnavailableException

- A subclass of ServletException
- Indicates that the **servlet is not available**

Reading Servlet Parameters

The **ServletRequest** interface provides methods to read the names and values of parameters that are sent by the client in a request. These parameters are usually sent through an HTML form.

Methods:

1. getParameter(String name)

Returns the value of a parameter sent by the client.

Syntax: String value = request.getParameter("name");

SBC Karkala

2. getParameterNames()

Returns all parameter names as an enumeration.

3. getParameterValues(String name)

Returns all values of a parameter (used when multiple values exist).

The javax.servlet.http Package

The **javax.servlet.http** package contains a number of interfaces and classes that are commonly used by servlet developers. You will see that its functionality makes it easy to build servlets that work with HTTP requests and responses.

The following table summarizes the core interfaces that are provided in this package:

Interface	Description
HttpServletRequest	Enables servlets to read data from an HTTP request.
HttpServletResponse	Enables servlets to write data to an HTTP response.
HttpSession	Allows session data to be read and written.
HttpSessionBindingListener	Informs an object that it is bound to or unbound from a session.

The following table summarizes the core classes that are provided in this package. The most important of these is **HttpServlet**. Servlet developers typically extend this class in order to process HTTP requests.

Class	Description
Cookie	Allows state information to be stored on a client machine.
HttpServlet	Provides methods to handle HTTP requests and responses.
HttpSessionEvent	Encapsulates a session-changed event.
HttpSessionBindingEvent	Indicates when a listener is bound to or unbound from a session value, or that a session attribute changed.

The HttpServletRequest Interface

The **HttpServletRequest** interface enables a servlet to obtain information about a client request. Several of its methods are shown in Table 31-5.

Method	Description
String getMethod()	Returns the HTTP method (such as GET, POST) used in the request.
HttpSession getSession()	Returns the current session associated with the request. If no session exists, a new session is created and returned.

TABLE 31-5 Various Methods Defined by **HttpServletRequest**

HttpServletResponse Interface

- It is used by a servlet to **send response to the client (browser)**.
- It helps in **creating and controlling the HTTP response**.
- It contains **status codes** like:

- SC_OK → request successful
- SC_NOT_FOUND → resource not found

Method	Description
void setHeader(String field, String value)	Adds the header field with value equal to value.
void setStatus(int code)	Sets the status code for this response to code.

TABLE 31-6 Various Methods Defined by **HttpServletResponse** *(continued)*

The HttpSession Interface

The **HttpSession** interface enables a servlet to read and write the state information that is associated with an HTTP session. Several of its methods are summarized in Table 31-7. All of these methods throw an **IllegalStateException** if the session has already been invalidated.

Method	Description
Object getAttribute(String attr)	Returns the value associated with the name passed in attr. Returns null if attr is not found.
void removeAttribute(String attr)	Removes the attribute specified by attr from the session.

TABLE 31-7 The Methods Defined by **HttpSession**

The HttpSessionBindingListener:

It has two methods:

void valueBound(HttpSessionBindingEvent e)

- This method is invoked automatically when an object is **added** to an HTTP session

void valueUnbound(HttpSessionBindingEvent e)

- This method is invoked when an object is **removed** from a session.

The Cookie Class

 A **Cookie** is a small piece of data stored on the **client (browser)**.

 It is helpful to **track user activities** and **remember user information** (like login details) for future visits.

What information can be stored in a cookie?

- Name of the cookie
- Value of the cookie
- Expiry time (maximum age)
- Domain and path of the cookie

Method	Description
String getName()	Returns the name of the cookie.
String getValue()	Returns the value stored in the cookie.
void setValue(String v)	Sets/updates the value of the cookie.
void setMaxAge(int secs)	Sets how long the cookie is stored (in seconds).

TABLE 31-8 The Methods Defined by **Cookie**

The HttpServlet Class

The **HttpServlet** class extends **GenericServlet**. It is commonly used when developing servlets that receive and process HTTP requests. The methods of the **HttpServlet** class are summarized in Table 31-9.

Method	Description
void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP GET request.
void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException	Handles an HTTP POST request.

TABLE 31-9 The Methods Defined by **HttpServlet**

HttpSessionEvent:

- It is a class used to represent **session-related events**.
- It is created when there is a **change in a session** (like creation or destruction).

Constructor: HttpSessionEvent(HttpSession session)
Method: HttpSession getSession()
Meaning: Returns the session where the event occurred.

HttpSessionBindingEvent:

- It is a class used for **session attribute binding/unbinding events**.
- It occurs when an object is **added or removed** from a session.

Constructors:
HttpSessionBindingEvent(HttpSession session, String name)
HttpSessionBindingEvent(HttpSession session, String name, Object val)
Methods:
String getName() - returns attribute name
HttpSession getSession() - returns the session
Object getValue() - returns the attribute value

SBC Karkala

Handling HTTP Requests and Responses

The **HttpServlet** class provides specialized methods to handle different types of HTTP requests. A servlet developer overrides these methods to process client requests. The important methods include **doGet(), doPost(), doDelete(), doPut(), doHead(), doOptions(), and doTrace()**. Among these, **GET and POST** are commonly used for handling form data.

doGet() Method in Servlet

Purpose:

- The **doGet()** method is used to handle **HTTP GET requests**.
- It is mainly used when data is sent from the client to the server through the **URL (query string)**.

Behavior

- Called automatically when the client sends a **GET request**
- Reads input using request.getParameter()
- Sends response to browser using PrintWriter
- Data is visible in the **URL**

HTML Code (ColorGet.htm)

```
<html>
<body>
<form action="ColorServlet" method="get">
   Enter Color: <input type="text" name="color">
   <input type="submit" value="Submit">
</form>
</body>
</html>
```

Servlet Code (ColorGetServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorServlet extends HttpServlet {
   public void doGet(HttpServletRequest request,
                     HttpServletResponse response)
          throws IOException {
      String color = request.getParameter("color");
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("Selected Color: " + color);
   }
}
```

SBC Karkala

Explanation

- The doGet() method handles GET requests.
- getParameter() reads the selected color.
- PrintWriter is used to send output to the browser.
- The response displays the selected color.

---

doPost() Method in Servlet

Purpose

The **doPost()** method is used to handle **HTTP POST requests**.
It is used when data is sent through the **request body**, so the data is **not visible in the URL**.

Behavior

- Invoked when the client submits a form using **method="post"**
- Reads input using request.getParameter()
- Sends response using PrintWriter
- Data is **secure compared to GET** (not shown in URL)

HTML Code (ColorPost.htm)

```
<html>
<body>
<form action="ColorServlet" method="post">
   Enter Color: <input type="text" name="color">
   <input type="submit" value="Submit">
</form>
</body>
</html>
```

Servlet Code (ColorPostServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorPostServlet extends HttpServlet {
   public void doPost(HttpServletRequest request,
                      HttpServletResponse response)
          throws IOException {
      String color = request.getParameter("color");
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("Selected Color: " + color);
   }
}
```

SBC Karkala

Explanation

- The doPost() method handles POST requests.
- getParameter() reads the selected value.
- Output is generated using PrintWriter.
- The selected color is displayed in the browser.

Difference Between GET and POST

- **GET**: Data is sent through URL (visible as query string)
- **POST**: Data is sent in request body (not visible in URL)

Handling Cookies in Servlets

Cookies are small pieces of data stored on the client machine. They are used to maintain state information and track user activities. In servlets, cookies can be **created, added to the response, and later read from the request**.

Steps to Handle Cookies in Servlets

1. Creating a Cookie

- A cookie is created using the **Cookie class**:

```
Cookie ck= new Cookie("name", "value");
```

2. Sending Cookie to Client

- The cookie is sent to the browser using:

```
response.addCookie(ck);
```

- The cookie is added to the **HTTP response header**.

3. Reading Cookies from Client

- Cookies sent by the browser are read using:

```
Cookie[] cks= request.getCookies();
```

4. Accessing Cookie Data

- Cookie details are accessed using:

```
ck.getName();   // returns name
ck.getValue();  // returns value
```

5. Setting Expiry Time

- Cookie lifetime can be controlled using:

```
ck.setMaxAge(seconds);
```

- If not set, the cookie is deleted when the browser closes.

Example: Demonstrates cookie handling.

```
import java.io.*;
```

SBC Karkala

```
import javax.servlet.*;
import javax.servlet.http.*;
public class CookieDemo extends HttpServlet {
   public void doGet(HttpServletRequest req, HttpServletResponse res)
         throws IOException {
      Cookie c = new Cookie("name", "John");   // Create cookie
      res.addCookie(c);                        // Send cookie
      Cookie[] cookies = req.getCookies();     // Read cookies
      res.setContentType("text/html");         // Display
      PrintWriter out = res.getWriter();
      for (Cookie ck : cookies) {
         out.println(ck.getName() + " = " + ck.getValue() + "<br>");
      }
   }
}
```

Session Tracking

HTTP is a **stateless protocol**, so sessions are used to maintain user data across multiple requests. A session is created using the getSession() method, which returns an HttpSession object. Session data is shared among all servlets for the same client.

Methods and Their Purpose

- **HttpSession getSession()**

  - Creates a new session if none exists, otherwise returns the existing session.
- **void setAttribute(String name, Object value)**

  - Stores data in the session as a **name–value pair**.
- **Object getAttribute(String name)**

  - Retrieves the value associated with the given name.
- **Enumeration getAttributeNames()**

  - Returns all attribute names stored in the session.
- **void removeAttribute(String name)**

  - Removes a specific attribute from the session.

SBC Karkala

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class SimpleSession extends HttpServlet {
   public void doGet(HttpServletRequest request, HttpServletResponse response)
         throws IOException {
      HttpSession session = request.getSession();          // get or create session
      Integer count = (Integer) session.getAttribute("count");         // get previous value
      if (count == null)
        {
                count = 1;
        }
       else {
                count++;
            }

      session.setAttribute("count", count);    // store updated value
      // display result
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("Visit Count: " + count);
   }
}
```

- Uses session to **store visit count**
- First visit → count = 1
- Next visits → count increases

SBC Karkala

Unit 4: Java Server Page(JSP)

What is JSP?

Java Server Pages (JSP) is a server-side technology used to create dynamic web pages.
It allows embedding Java code inside HTML using special tags like <% %>.
A JSP page is converted into a servlet and works as a user interface for web applications.
It combines HTML, XML, and Java code in a single file.
JSP is used to collect user input through forms and display data from databases.
It helps in generating dynamic content for web pages.
JSP also supports accessing JavaBeans, passing control between pages, and sharing data between requests and pages.

Simple Example

```
<html>
<body>
<%
   out.println("Welcome to JSP!");
%>
</body>
</html>
```

Why Use JSP?

[Explain why JSP is a compelling choice for web development compared to the Common Gateway Interface (CGI).]

Java Server Pages often serve the same purpose as programs implemented using the Common Gateway Interface (CGI). But JSP offers several advantages in comparison with the CGI.

- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having separate CGI files.
- JSP are always compiled before they are processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.

- Java Server Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP, etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.

Finally, JSP is an integral part of Java EE, a complete platform for enterprise class applications. This means that JSP can play a part in the simplest applications to the most complex and demanding.

Advantages of JSP:

vs. Active Server Pages (ASP):
JSP uses Java, so it is more powerful and not limited to Microsoft languages. JSP is portable and can run on different operating systems and servers.

vs. Pure Servlets:
JSP is easier to write and modify because HTML can be written directly without many println statements.

vs. Server-Side Includes (SSI):
JSP supports complex applications, while SSI is suitable only for simple tasks.

vs. JavaScript:
JSP can interact with the server and database, while JavaScript mainly works on the client side.

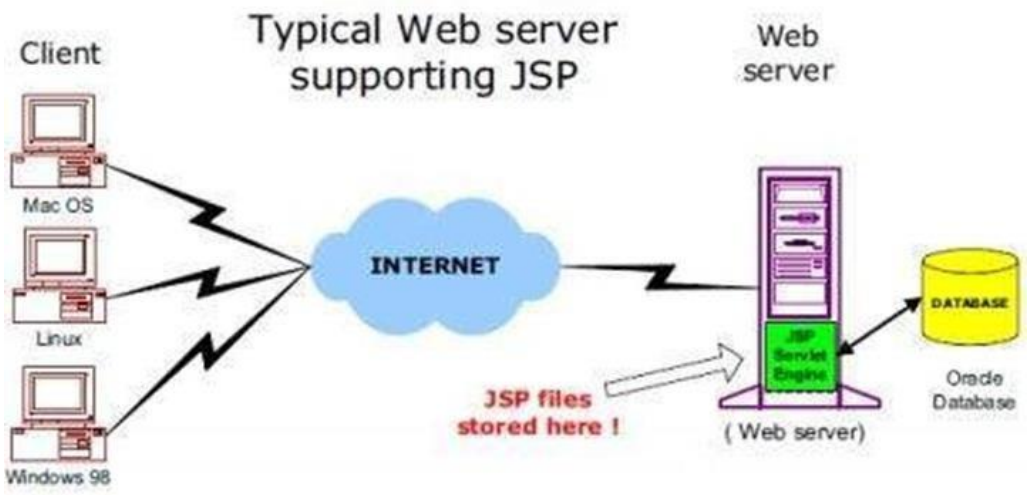
vs. Static HTML:
JSP can generate dynamic content, while static HTML cannot.

JSP - Architecture

The web server needs a JSP engine, i.e., a container to process JSP pages. The JSP container is responsible for intercepting requests for JSP pages. This material makes use of Apache which has built-in JSP container to support JSP pages development.

A JSP container works with the Web server to provide the runtime environment and other services a JSP needs. It knows how to understand the special elements that are part of JSPs.

Following diagram shows the position of JSP container and JSP files in a Web application.



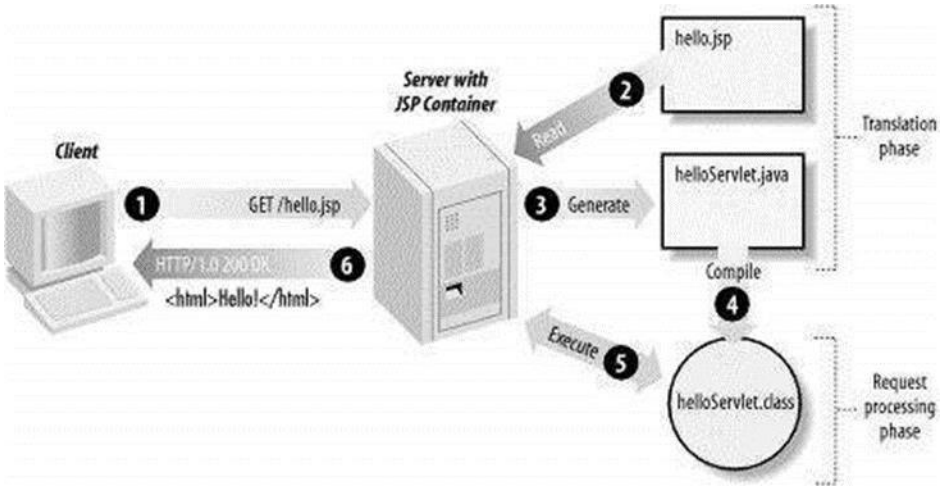
JSP Processing

The following steps explain how the web server creates the Web page using JSP:

- As with a normal page, browser (client) sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with **.jsp** instead of **.html**.
- The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to **println( )** statements and all JSP elements are converted to Java code. This code implements the corresponding dynamic behavior of the page.
- The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
- Apart of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format. This output is further passed on to the web server by the servlet engine inside an HTTP response.
- The web server forwards the HTTP response to our browser in terms of static HTML content.

- Finally, the web browser handles the dynamically-generated HTML page inside the HTTP response exactly as if it were a static page.

All the above mentioned steps canbe seen in the following diagram:



Typically, the JSP engine checks to see whether a servlet for a JSP file already exists and whether the modification date on the JSP is older than the servlet. If the JSP is older than its generated servlet, the JSP container assumes that the JSP hasn't changed and that the generated servlets till matches the JSP's contents. This makes the process more efficient than with the other scripting languages (such as PHP) and therefore faster.

So in a way, a JSP page is really just another way to write a servlet without having to be a Java programming wiz. Except for the translation phase, a JSP page is handled exactly like a regular servlet.

JSP life cycle

The key to understanding the low-level functionality of JSP is to understand the simple life cycle they follow.

A JSP life cycle is defined as the process from its creation till the destruction. This is similar to a servlet life cycle with an additional step which is required to compile a JSP into servlet.

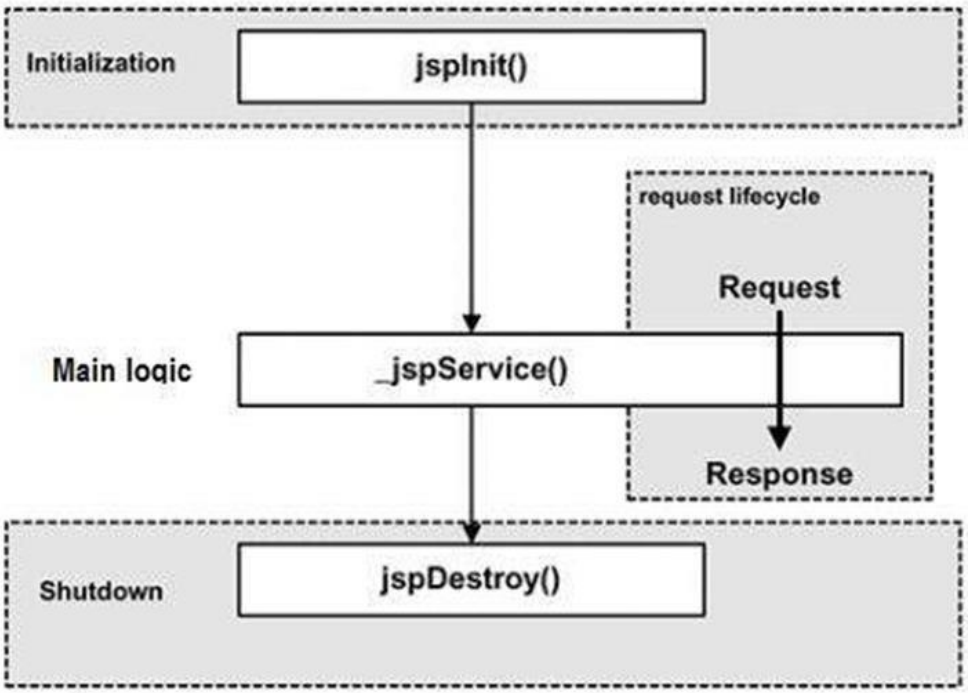
Paths Followed By JSP

The following are the paths followed by a JSP

- Compilation

- Initialization
- Execution
- Cleanup

The four major phases of a JSP life cycle are very similar to the Servlet Life Cycle. The four phases have been described below:



JSP Compilation

When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page. If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page.

The compilation process involves three steps:

- Parsing the JSP.
- Turning the JSP in to a servlet.
- Compiling the servlet.

JSP Initialization

When a container loads a JSP it invokes the jspInit() method before servicing

any requests**.** If we need to perform JSP-specific initialization, override the jspInit() method:

```
Public void jspInit(){
  //Initializationcode...
}
```

Typically, initialization is performed only once and as with the servlet init method, we generally initialize database connections, open files, and create lookup tables in the jspInit method.

JSP Execution

This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed.

Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the **_jspService()** method in the JSP.

The_jspService() method takes an **HttpServletRequest** and an **HttpServletResponse** as its parameters as follows:

```
Void _jspService(HttpServletRequest request, HttpServletResponse response)
{
   //Servicehandlingcode...
}
```

The **_jspService()** method of a JSP is invoked on request basis. This is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods, i.e., **GET**, **POST**, **DELETE**, etc.

JSP Cleanup

The destruction phase of the JSP lifecycle represents when a JSP is being removed from use by a container.

The **jspDestroy()** method is the JSP equivalent of the destroy method for servlets. Override jspDestroy when you need to perform any cleanup, such as releasing database connections or closing open files.

The jspDestroy() method has the following form:

```
public void jspDestroy()
{
   //Yourcleanupcodegoeshere.
}
```

JSP Syntax:
Let us understand the basic use of simple syntax (i.e., elements) involved with JSP development.

Elements of JSP

The elements of JSP have been described below:

The Scriptlet

A scriptlet can contain any number of JAVA language statements, variable or method declarations, or expressions that are valid in the page scripting language.

Following is the syntax of Scriptlet:

```
<% code fragment %>
```

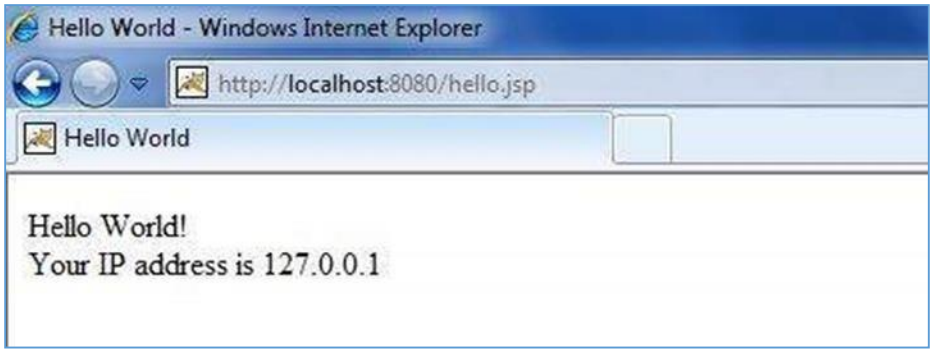
We can write the XML equivalent of the above syntax as follows:

```
<jsp:scriptlet>
   Code fragment
</jsp:scriptlet>
```

Any text, HTML tags, or JSP elements we write must be outside the scriptlet. Following is the simple and first example for JSP:

```
<html>
<head><title>HelloWorld</title></head>
<body>
HelloWorld!<br/>
<%
out.println("Your IP address is"+request.getRemoteAddr());
%>
</body>
</html>
```

Let us keep the above code in JSP file **hello.jsp** and put this file in**C:\apache-tomcat- 7.0.2\webapps\ROOT** directory. Browse through the same using URL **http://localhost:8080/hello.jsp**. This would generate the following result:



JSP Declarations

A declaration declares one or more variables or methods that we can use in Java code later in the JSP file. You must declare the variable or method before you use it in the JSP file.

Following is the syntax for JSP Declarations:

```
<%! declaration; [declaration;]+... %>
```

You can write the XML equivalent of the above syntax as follows:

```
<jsp:declaration>
   code fragment
</jsp:declaration>
```

Following is an example for JSP Declarations:

```
<%! int i=0;%>
<%! int a,b,c;%>
<%! Circle a=new Circle(2.0); %>
```

JSP Expression

A JSP expression element contains a scripting language expression that is evaluated, converted to a String, and inserted where the expression appears in the JSP file. Because the value of an expression is converted to a String, you can use an expression within a line of text, whether or not it is tagged with HTML, in a JSP file.

The expression element can contain any expression that is valid according to the Java Language Specification but you cannot use a semicolon to end an expression.

Following is the syntax of JSP Expression:

```
<%= expression %>
```

You can write the XML equivalent of the above syntax as follows:

```
<jsp:expression>
   expression
</jsp:expression>
```

Following example shows a JSP Expression:

```
<html>
<head><title>A Comment Test</title></head>
<body>
<p>
   Today'sdate:<%= (new java.util.Date()).toLocaleString()%>
</p>
</body>
</html>
```

The above code will generate the following result:

```
Today's date: 11-Sep-2010 21:24:25
```

JSP Comments

JSP comment marks the text or the statements that the JSP container should ignore. A JSP comment is useful when you want to hide or "comment out", a part of your JSP page.

Following is the syntax of the JSP comments:

```
<%-- This is JSP comment-- %>
```

Following example shows the JSP Comments:

```
<html>
<head><title>A Comment Test</title></head>
<body>
<h2>A Test of Comments</h2>
<%--This comment will not be visible in the page source--%>
</body>
</html>
```

The above code will generate the following result:

```
A Test of Comments
```

There are a small number of special constructs you can use in various cases to insert comments or characters that would otherwise be treated specially. Here's a summary:

Syntax	**Purpose**
<%-- comment --%>	A JSP comment. Ignored by the JSP engine.

<!--comment-->	An HTML comment. Ignored by the browser.
<\%	Represents static **<%** literal.
%\>	Represents static **%>** literal.
\'	A single quote in an attribute that uses single quotes.
\"	A double quotein an attribute that uses double quotes.

JSP Directives

A JSP directive affects the overall structure of the servlet class. It usually has the following form:

```
<%@directive attribute="value"%>
```

There are three types of directive tag:

Directive	**Description**
<%@ page... %>	Defines page-dependent attributes, such as scripting language, error page and buffering requirements.
<%@ include... %>	Includes a file during the translation phase.
<%@ taglib... %>	Declares a tag library, containing custom actions, used in the page

JSP Actions

JSP actions use **constructs** in XML syntax to control the behavior of the servlet engine. You can dynamically insert a file, reuse JavaBeans components, forward the user to another page, or generate HTML for the Java plugin.

There is only one syntax for the Action element, as it conforms to the XML standard:

```
<jsp:action_name attribute="value"/>
```

Action elements are basically predefined functions. Following table lists out the available JSP Actions:

Syntax	**Purpose**
jsp:include	Includes a file at the time the page is requested.
jsp:useBean	Finds or instantiates a JavaBean.
jsp:setProperty	Sets the property of a JavaBean.
jsp:getProperty	Inserts the property of a JavaBean into the output.
jsp:forward	Forwards the requester to a new page.
jsp:plugin	Generates browser-specific code that makes an OBJECT or EMBED tag for the Java plugin.
jsp:element	Defines XML elements dynamically.
jsp:attribute	Defines dynamically-defined XML element's attribute.
jsp:body	Defines dynamically-defined XML element's body.
jsp:text	Used to write template text in JSP pages and documents.

JSP Implicit Objects

JSP supports nine automatically defined variables, which are also called implicit objects. These variables are:

Objects	**Description**
request	This is the **HttpServletRequest** object associated with the request.

response	This is the **HttpServletResponse** object associated with the response to the client.
out	This is the **PrintWriter** object used to send output to the client.
session	This is the **HttpSession** object associated with the request.
application	This is the **ServletContext** object associated with application context.
config	This is the **ServletConfig** object associated with the page.
pageContext	This encapsulates use of server-specific features like higher performance **JspWriters**.
page	This is simply a synonym for **this**, and is used to call the methods defined by the translated servlet class.
Exception	The **Exception** object allows the exception data to be accessed by designated JSP.

Control-Flow Statements

You can use all the APIs and building blocks of Java in your JSP programming including decision-making statements, loops, etc.

Decision-Making Statements

The **if...else** block starts out like an ordinary Scriptlet, but the Scriptlet is closed at each line with HTML text included between the Scriptlet tags.

```
<%!intday=3;%>
<html>
<head><title>IF...ELSEExample</title></head>
<body>
<%if(day==1|day==7){%>
      <p>Todayisweekend</p>
<%}else{%>
      <p>Todayisnotweekend</p>
<%}%>
</body>
</html>
```

The above code will generate the following result:

```
Today is not weekend
```

Now look at the following **switch...case** block which has been written a bit differently using **out.println()** and inside Scriptletas:

```
<%! int day = 3; %>
<html>
<head><title>SWITCH...CASE Example</title></head>
<body>
<%
switch(day) {
case 0:
   out.println("It\'s Sunday.");
```

```
   break;
case 1:
   out.println("It\'sMonday.");
   break;
case2:
   out.println("It\'sTuesday.");
   break;
case3:
   out.println("It\'sWednesday.");
   break;
case4:
   out.println("It\'sThursday.");
   break;
case5:
   out.println("It\'sFriday.");
   break;
default:
   out.println("It'sSaturday.");
}
%>
</body>
</html>
```

The above code will generate the following result:

```
It's Wednesday.
```

Loop Statements

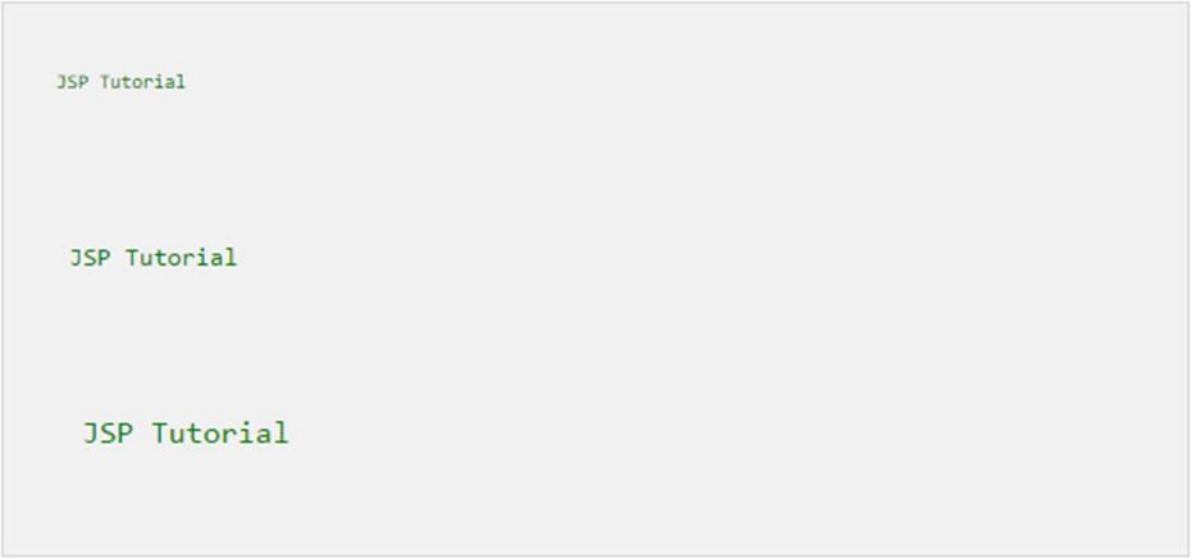
You can also use three basic types of looping blocks in Java: **for, while,** and **do...while** blocks in your JSP programming.

Let us look at the following **for** loop example:

```
<%! int fontSize; %>
```

```
<html>
<head><title>FOR LOOP Example</title></head>
<body>
<%for(fontSize=1;fontSize<=3;fontSize++){ %>
   <fontcolor="green"size="<%=fontSize%>"> JSP
    Tutorial
   </font><br/>
<%}%>
</body>
</html>
```

The above code will generate the following result:

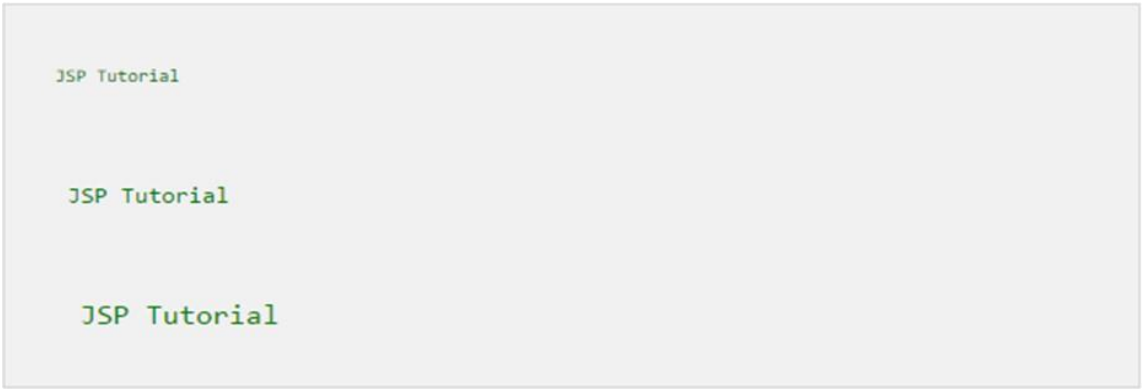


Above example can be written using the **while** loop as follows:

```
<%! int fontSize; %>
<html>
<head><title>WHILE LOOP Example</title></head>
<body>
<%while(fontSize<=3){ %>
   <fontcolor="green"size="<%=fontSize%>"> JSP
    Tutorial
```

```
   </font><br/>
<%fontSize++;%>
<%}%>
</body>
</html>
```

The above code will generate the following result:



JSP Operators

JSP supports all the logical and arithmetic operators supported by Java. Following table lists out all the operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom.

Within an expression, higher precedence operators will be evaluated first.

Category	**Operator**	**Associativity**
Postfix	() [] .(dot operator)	Left to right
Unary	++ -- ! ~	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	>> >>> <<	Left to right

Relational	> >= < <=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= |=	Right to left
Comma	,	Left to right

JSP Literals

The JSP expression language defines the following literals:

- **Boolean:** true and false
- **Integer:** as in Java
- **Floating point:** as in Java
- **String:** with single and double quotes; " is escaped as \", ' is escaped as \', and \ is escaped as \\.
- **Null:** null

JSP Directives

The JSP directives provide directions and instructions to the container, telling it how to handle certain aspects of the JSP processing.

A JSP directive affects the overall structure of the servlet class. It usually has the following form:

```
<%@ directive attribute="value" %>
```

Directives can have a number of attributes which you can list down as key-value pairs and separated by commas.

The blanks between the @symbol and the directive name, and between the last attribute and the closing %>, are optional.

There are three types of directive tag:

Directive	Description
<%@ page... %>	Defines page-dependent attributes, such as scripting language, error page, and buffering requirements.
<%@ include... %>	Includes a file during the translation phase.
<%@ taglib...%>	Declares a tag library, containing custom actions, used in the page

The page Directive

The **page** directive is used to provide instructions to the container. These instructions pertain to the current JSP page. You may code page directives anywhere in your JSP page. By convention, page directives are coded at the top of the JSP page.

Following is the basic syntax of the page directive:

```
<%@ page attribute="value" %>
```

You can write the XML equivalent of the above syntax as follows:

```
<jsp:directive.page attribute="value"/>
```

Attributes

Following table lists out the attributes associated with page directive:

Attribute	Purpose
buffer	Specifies a buffering model for the output stream.
autoFlush	Controls the behavior of the servlet output buffer.
contentType	Defines the character encoding scheme.
errorPage	Defines the URL of another JSP that reports on Java unchecked runtime exceptions.
isErrorPage	Indicates if this JSP page is a URL specified by another JSP page's errorPage attribute.
extends	Specifies a superclass that the generated servlet must extend.
import	Specifies a list of packages or classes for use in the JSP as the Java import statement does for Java classes.
info	Defines a string that can be accessed with the servlet's **getServletInfo()** method.
isThreadSafe	Defines the threading model for the generated servlet.
language	Defines the programming language used in theJSP page.
session	Specifies whether or not the JSP page participates in HTTP sessions
isELIgnored	Specifies whether or not the EL expression within the JSP page will be ignored.
isScriptingEnabled	Determines if the scripting elements are allowed for use.

The buffer Attribute

The **buffer** attribute specifies the buffering characteristics for the server output response object.

You may code a value of "**none**" to specify no buffering so that the servlet output is immediately directed to the response object or you may code a maximum buffer size in kilobytes, which directs the servlet to write to the buffer before writing to the response object.

To direct the servlet to write the output directly to the **response output object**, use the following:

```
<%@ page buffer="none" %>
```

Use the following to direct the servlet to write the output to a buffer of size not less than 8 kilobytes:

```
<%@ page buffer="8kb" %>
```

The autoFlush Attribute

The autoFlush attribute specifies whether buffered output should be flushed automatically when the buffer is filled, or whether an exception should be raised to indicate the buffer overflow.

A value of **true (default)** indicates automatic buffer flushing and a value of false throws an exception.

The following directive causes the servlet to throw an exception when the servlet's output buffer is full:

```
<%@ page autoFlush="false" %>
```

This directive causes the servlet to flush the output buffer when full:

```
<%@ page autoFlush="true" %>
```

Usually, the buffer and the autoFlush attributes are coded on a single page directive as follows:

```
<%@page buffer="16kb" autoflush="true" %>
```

The contentType Attribute

The contentType attribute sets the character encoding for the JSP page and for the generated response page. The default content type is **text/html**, which is the standard content type for HTML pages.

If you want to write out XML from your JSP, use the following page directive:

```
<%@ page contentType="text/xml" %>
```

The following statement directs the browser to render the generated page as HTML:

```
<%@ page contentType="text/html" %>
```

The following directive sets the content type as a Microsoft Word document:

```
<%@ page contentType="application/msword" %>
```

You can also specify the character encoding for the response. For example, if you wanted to specify that the resulting page that is returned to the browser uses **ISOLatin1**, you can use the following page directive:

```
<%@ page contentType="text/html:charset=ISO-8859-1" %>
```

The errorPage Attribute

The **errorPage** attribute tells the JSP engine which page to display if there is an error while the current page runs. The value of the errorPage attribute is a relative URL.

The following directive displays MyErrorPage.jsp when all uncaught exceptions are thrown:

```
<%@ page errorPage="MyErrorPage.jsp" %>
```

The isErrorPage Attribute

The **isErrorPage** attribute indicates that the current JSP can be used as the error page for another JSP.

The value of isErrorPage is either true or false. The default value of the isErrorPage attribute is false.

For example, the **handleError.jsp** sets the isErrorPage option to true because it is supposed to handle errors:

```
<%@ page isErrorPage="true" %>
```

The extends Attribute

The **extends** attribute specifies a super class that the generated servlet must extend.

For example, the following directive directs the JSP translator to generate the servlet such that the servlet extends ***somePackage.SomeClass***:

```
<%@ page extends="somePackage.SomeClass" %>
```

The import Attribute

The **import** attribute serves the same function as, and behaves like, the Java import statement. The value for the import option is the name of the package you want to import.

To import **java.sql.***, use the following page directive:

```
<%@ page import="java.sql.*" %>
```

To import multiple packages, you can specify them separated by comma as follows:

```
<%@ page import="java.sql.*, java.util.*" %>
```

By default, a container automatically imports **java.lang.***, **javax.servlet.***, **javax.servlet.jsp.***, and **javax.servlet.http.***.

The info Attribute

The **info** attribute lets you provide a description of the JSP. The following is a coding example:

```
<%@ page info="ThisJSPPageWrittenByZARA" %>
```

The isThreadSafe Attribute

The **isThreadSafe** option marks a page as being thread-safe. By default, all JSPs are considered thread-safe. If you set the isThreadSafe option to false, the

JSP engine makes sure that only one thread at a time is executing your JSP.

The following page directive sets the **isThreadSafe** option to false:

```
<%@ page isThreadSafe="false" %>
```

The language Attribute

The **language** attribute indicates the programming language used in scripting the JSP page.

For example, because you usually use Java as the scripting language, your language option looks like this:

```
<%@ page language="java" %>
```

The session Attribute

The **session** attribute indicates whether or not the JSP page uses HTTP sessions. A value of true means that the JSP page has access to a built in **session** object and a value of false means that the JSP page cannot access the built in session object.

Following directive allows the JSP page to use any of the built in object session methods such as **session.getCreationTime()** or **session.getLastAccessTime()**:

```
<%@ page session="true" %>
```

The is ELIgnored Attribute

The **isELIgnored** attribute gives you the ability to disable the evaluation of Expression Language (EL) expressions which has been introduced in JSP 2.0.

The default value of the attribute is true, meaning that expressions, **${...}**, are evaluated as dictated by the JSP specification. If the attribute is set to false, then expressions are not evaluated but rather treated as static text.

Following directive sets an expression not to be evaluated:

```
<%@ page isELIgnored="false" %>
```

The isScriptingEnabled Attribute

The **isScriptingEnabled** attribute determines if the scripting elements are allowed for use.

The **default value (true)** enables scriptlets, expressions, and declarations. If the attribute's value is set to false, a translation-time error will be raised if the JSP uses any scriptlets, expressions (non-EL), or declarations.

The attribute's value can be set to false if you want to restrict the usage of scriptlets, expressions (non-EL), or declarations:

```
<%@ page isScriptingEnabled="false" %>
```

The include Directive

The **include** directive is used to include a file during the translation phase.This directive tells the container to merge the content of other external files with the current JSP during the translation phase. You may code the ***include*** directives anywhere in your JSP page.

The general usage form of this directive is as follows:

```
<%@ include file="relativeurl">
```

The filename in the include directive is actually a relative URL. If you just specify a filename with no associated path, the JSP compiler assumes that the file is in the same directory as your JSP.

You can write the XML equivalent of the above syntax as follows:

```
<jsp:directive.include file="relativeurl" />
```

Example

A good example of the **include** directive is including a common header and footer with multiple pages of content.

Let us define following three files **(a)header.jps**, **(b)footer.jsp**, and **(c)main.jsp** as follows:

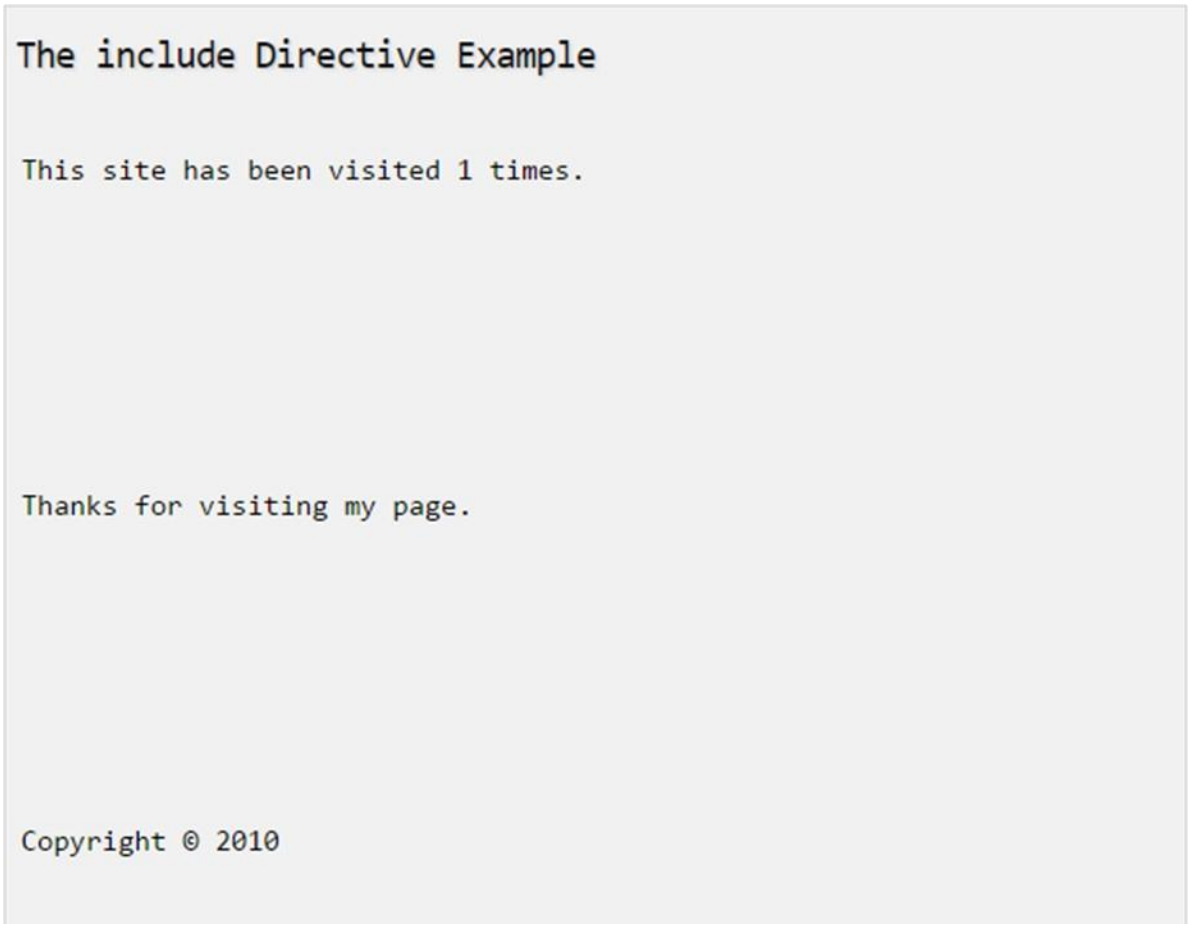
Followingisthecontentof**header.jsp**:

```
<%!
 intpageCount=0; void
 addCount() {
   pageCount++;
 }
%>
<%addCount();%>
<html>
<head>
<title>TheincludeDirectiveExample</title>
</head>
<body>
<center>
<h2>TheincludeDirectiveExample</h2>
<p>Thissitehasbeenvisited<%=pageCount%>times.</p>
</center>
<br/><br/>
```

Following is the content of **footer.jsp**:

```
<%@includefile="header.jsp"%>
<center>
<p>Thanksforvisitingmypage.</p>
</center>
<%@includefile="footer.jsp"%>
```

Let us now keep all these files in the root directory and try to access **main.jsp**. You will receive the following output:



Refresh **main.jsp** and you will find that the page hit counter keeps increasing.

You can design your web pages based on your creative instincts; it is recommended you keep the dynamic parts of your website in separate files and then include them in the main file. This makes it easy when you need to change a part of your webpage.

The taglib Directive

The Java Server Pages API allow you to define custom JSP tags that look like HTML or XML tags and a tag library is a set of user-defined tags that implement custom behavior.
The **taglib** directive declares that your JSP page uses a set of custom tags, identifies the location of the library, and provides means for identifying the custom tags in your JSP page.

The taglib directive follows the syntax given below:

```
<%@ taglib uri="uri" prefix="prefixOfTag">
```

Here, the **uri** attribute value resolves to a location the container understands and the **prefix** attribute informs a container what bits of markup are custom actions.

You can write the XML equivalent of the above syntax as follows:

```
<jsp:directive.taglib uri="uri" prefix="prefixOfTag"/>
```

When you use a custom tag, it is typically of the form **<prefix:tagname>**. The prefix is the same as the prefix you specify in the taglib directive, and the tag name is the name of a tag implemented in the tag library.

Example

For example, suppose the **custlib** tag library contains a tag called **hello**. If you wanted to use the hello tag with a prefix of **mytag**, your tag would be **<mytag:hello>** and it will be used in your JSP file as follows:

```
<%@ taglib uri="http://www.example.com/custlib" prefix="mytag" %>
<html>

<body>

<mytag:hello/>

</body>
</html>
```

We can call another piece of code using **<mytag:hello>**.

The Concept of JDBC

JDBC (Java Database Connectivity) is an API that allows Java programs to interact with databases. It was created to standardize database access across different DBMSs, making Java programs database-independent.

JDBC Driver Types

JDBC drivers are software components that enable Java applications to communicate with a database management system (DBMS). Based on their working mechanism, JDBC drivers are classified into four types.

Type 1: JDBC–ODBC Bridge Driver

The Type 1 driver is also known as the JDBC–ODBC Bridge. It converts JDBC method calls into ODBC function calls. The ODBC driver further converts these calls into the database-specific format understood by the DBMS.

This driver introduces multiple layers of translation: JDBC → ODBC → DBMS

Because of this additional translation layer, performance may be affected. It is generally not recommended for large-scale or mission-critical applications. However, it can be useful for simple applications and testing purposes.

Type 2: Java / Native Code Driver

The Type 2 driver uses Java along with native code libraries provided by the database vendor. It converts JDBC calls into database-specific native API calls.

This driver is dependent on the platform because it requires native libraries specific to the DBMS. As a result, portability is reduced. Also, separate drivers are required for different databases.

Despite this limitation, it offers better performance than Type 1 since it eliminates one level of translation.

Type 3: JDBC Network Protocol Driver

The Type 3 driver uses an intermediate server (middleware). It converts JDBC calls into a database-independent protocol, which is then translated by the middleware into the DBMS-specific protocol.

This approach allows communication with multiple databases using a single driver. It improves flexibility and scalability.

However, since it involves network communication with a middleware server, it may introduce additional overhead and latency.

Type 4: JDBC Thin Driver

The Type 4 driver is a pure Java driver that directly converts JDBC calls into the database-specific protocol. It communicates directly with the DBMS without requiring any intermediate layers.

Since there is no extra translation layer, it provides the best performance among all driver types. It is platform-independent and widely used in real-world applications.

SBC Karkala

This driver is considered the most efficient and fastest way to interact with a database using JDBC.

JDBC Packages

The JDBC API is mainly contained in two packages:

java.sql

This is the core package of JDBC and is part of J2SE. It contains essential interfaces and classes required for database connectivity and operations.

It provides functionalities such as:

- Establishing database connections
- Executing SQL queries
- Handling results from the database

Key components like Connection, Statement, and ResultSet belong to this package.

javax.sql

This package extends the functionality of java.sql and provides advanced features. It is commonly used in enterprise-level applications.

It includes:

- Support for connection pooling
- Integration with Java Naming and Directory Interface (JNDI)
- Advanced data source management

This package enhances performance and scalability in database applications.

Overview of the JDBC Process

JDBC-based applications follow a standard sequence of operations to interact with a database. **This process consists of five main steps.**

1. Load the JDBC Driver

Before establishing a connection, the JDBC driver must be loaded into memory.

This is done using the Class.forName() method.

Loading the driver ensures that the application can communicate with the DBMS through the appropriate driver.

```
Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver");
```

2. Connect to the DBMS

After loading the driver, a connection is established using the DriverManager.getConnection() method.

This method requires: Database URL , Username , Password

It returns a Connection object, which acts as a link between the Java application and the database. All further operations are performed using this connection.

SBC Karkala

```
String url = "jdbc:odbc:CustomerInformation";
String userID = "jim";
String password = "keogh";
Statement DataRequest;
private Connection Db;
try {
   Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver");
   Db = DriverManager.getConnection(url,userID,password);
 }
```

3. Create and Execute an SQL Statement

Once the connection is established, SQL queries are sent to the database using a Statement object.

The createStatement() method is used to create this object. SQL queries are executed using methods such as executeQuery().

The result of a query is returned as a ResultSet object, which contains the data retrieved from the database.

```
Statement DataRequest;
ResultSet Results;
try {
      String query = "SELECT * FROM Customers";
      DataRequest = Db.createStatement();
      Results = DataRequest.executeQuery (query);
      DataRequest.close();
 }
```

4. Process Data Returned by the DBMS

The ResultSet object is used to access and manipulate the data returned by the DBMS. It maintains a cursor pointing to the current row of data. Methods like next() are used to move through the rows, and methods like getString() are used to retrieve column values.

This step allows the application to read and process the query results.

```
String FirstName;
String LastName;
String printrow;
boolean Records = Results.next();
if (!Records ) {
   System.out.println( "No data returned");
   return;
}
else
{
 do {
   FirstName = Results.getString (FirstName) ;
   LastName = Results.getString (LastName) ;
   printrow = FirstName + " " + LastName;
   System.out.println(printrow);
 } while ( Results.next() );
}
```

5. Terminate the Connection to the DBMS

After completing all database operations, the connection must be closed using the close() method.

Closing the connection releases system resources and ensures proper termination of database interaction.

It is also a good practice to close the ResultSet and Statement objects before closing the connection.

```
Db.close();
```

Database Connection

A J2ME application does not connect directly to the database management system (DBMS). Instead, it communicates with the DBMS through a **JDBC driver**. The JDBC driver acts as an intermediary between the Java application and the database.

Before establishing a connection, the JDBC driver must be **loaded and registered**. Loading the driver brings it into the **Java Virtual Machine (JVM)**, making it available for use. Once loaded, the driver is automatically registered with the DriverManager, which manages all database drivers.

The Class.forName() method is used to load the JDBC driver.

For example: Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

If the driver fails to load, a ClassNotFoundException is generated. Therefore, it is always handled using a try-catch block to ensure proper error handling.

```
try {
   Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver");
}
catch (ClassNotFoundException error) {
   System.err.println("Unable to load the JDBC/ODBC bridge." +
error.getMessage());
   System.exit(1);
}
```

The Connection

After successfully loading and registering the JDBC driver, the next step is to establish a connection with the database.

The database must be properly associated with the JDBC driver. This association is usually configured by the database administrator or system administrator. For learning purposes, students often use simple databases such as Microsoft Access.

The connection to the database is established using a **URL (Uniform Resource Locator)**. The JDBC URL defines how the application connects to the database.

A JDBC URL consists of three main parts:

- **jdbc** → Indicates that JDBC protocol is used
- **subprotocol** → Specifies the JDBC driver name
- **subname** → Specifies the database name

This URL is passed to the DriverManager.getConnection() method along with user credentials to establish the connection.

The connection created acts as a **communication link** between the Java application and the database. All database operations such as sending queries and receiving results are performed through this connection.

The connection to the database is established using the getConnection() methods of the DriverManager class. These methods request access to the database from the DBMS. The DBMS decides whether to grant or reject the connection request.

If access is granted, a **Connection object** is returned. If not, an **SQLException** is thrown, indicating that the connection failed.

Methods to Establish Connection

There are three commonly used versions of the getConnection() method.

1. Connection Using Only URL

In some cases, the database allows access without requiring authentication. In such situations, only the database URL is provided.

String url = "jdbc:odbc:CustomerInformation";

Db = DriverManager.getConnection(url);

Here, the DBMS identifies the database using the URL and grants access without user credentials.

---

2. Connection Using URL, User ID, and Password

Most databases restrict access and require authentication. In this case, the user must provide a **user ID and password** along with the URL.

```
String url = "jdbc:odbc:CustomerInformation";
String userID = "jim";
String password = "keogh";
Db = DriverManager.getConnection(url, userID, password);
```

This method ensures that only authorized users can access the database.

3. Connection Using Properties Object

Sometimes, additional information is required apart from the user ID and password. This information is called **properties**.

These properties are stored in a file and loaded into a Properties object. The object is then passed to the getConnection() method.

```
Properties props = new Properties();
props.load(new FileInputStream("DBProps.txt"));
Db = DriverManager.getConnection(url, props);
```

This approach provides flexibility to include multiple parameters required by the DBMS.

Associating the JDBC/ODBC Bridge with the Database

You use the ODBC Data Source Administrator to create the association between the database and the JDBC/ODBC Bridge. Here's what you need to do:

1. Select Start | Settings | Control Panel.
2. Select ODBC 32 to display the ODBC Data Source Administrator.
3. Add a new user by selecting the Add button.
4. Select the driver, and then select Finish. Use the Microsoft Access Driver if you are using Microsoft Access, otherwise select the driver for the DBMS that you are using. If you don't find the driver for your DBMS on the list, you'll need to install the driver. Contact the manufacturer of the DBMS for more information on how to obtain the driver.
5. Enter the name of the database as the Data Source name in the ODBC Microsoft Access Setup dialog box. This is the name that will be used within your Java database program to connect to the DBMS.
6. Enter a description for the data source. This is optional, but will be a reminder of the kind of data stored in the database.

7. Click the Select button. You'll be prompted to browse the directory of each hard drive connected to your computer in order to define the direct path to the database. Click OK once you locate the database, and the directory path and name of the database will be displayed in the ODBC Microsoft Access Setup dialog box.
8. Since this is your database, you can determine whether a login name and password are required to access the database.

   If so, click the Advanced button to display the Set Advanced Options dialog box. This dialog box is used to assign a login name (also referred to as a user ID) and a password to the database. Select OK. If not, skip this step.
9. When the ODBC Microsoft Access Setup dialog box appears, select OK.
10. Select OK to close the ODBC Data Source Administrator dialog box.

Timeout

In real-world applications, multiple clients or applications may try to access the database at the same time. This situation can lead to **performance degradation**, especially when the database is busy or connections are not readily available.
Instead of waiting indefinitely for the database to respond, a J2ME application can define a **timeout period**. If the DBMS does not respond within this time, the connection attempt is stopped.
The DriverManager class provides methods to handle this:

- DriverManager.setLoginTimeout(int seconds)
  → Sets the maximum time (in seconds) the DriverManager will wait for a connection.
- DriverManager.getLoginTimeout()
  → Returns the current timeout value set in seconds.

This mechanism helps improve application responsiveness by avoiding long delays when the database is overloaded or unavailable.

Connection Pool

Establishing a connection to a database is resource-intensive. Normally, each client must create its own connection, which can lead to two major problems:

- Keeping connections open blocks other clients if the number of connections is limited
- Repeatedly opening and closing connections increases time and reduces performance

To overcome these issues, **connection pooling** is used.

Concept of Connection Pool

A connection pool is a collection of pre-established database connections that are stored in memory. These connections are reused by different clients instead of creating new connections every time.

SBC Karkala

This improves performance and reduces overhead.
Clients do not directly manage these connections. Instead, they interact with the connection pool using the **DataSource interface**.

Types of Connections
There are two types of connections involved:
1. Physical Connection

- Created once by the application server
- Managed using PooledConnection objects
- Stored and reused in the connection pool

2. Logical Connection

- Requested by the client
- Obtained using DataSource.getConnection()
- Internally linked to an existing physical connection

Working of Connection Pool

- The application server creates a pool of physical connections
- These connections remain available in memory
- When a client requests a connection, a logical connection is provided
- After use, the logical connection is closed
- The physical connection remains in the pool for reuse

Accessing Connection Pool using JNDI
The connection pool is accessed using **Java Naming and Directory Interface (JNDI)**.
Steps involved:

1. Obtain the JNDI context
2. Lookup the connection pool using its name
3. Get a connection from the pool
4. Use the connection
5. Close the logical connection after use

```
Context ctext = new InitialContext();
DataSource pool = (DataSource) ctext.lookup("java:comp/env/jdbc/pool");
Connection db = pool.getConnection();
// interact with database
db.close();
```

Statement Objects
After establishing a connection with the database, the J2ME application needs to send SQL queries to access or modify data. For this purpose, JDBC provides different types of **Statement objects**.

SBC Karkala

There are three types of Statement objects:

- **Statement** → Executes queries immediately
- **PreparedStatement** → Executes precompiled queries
- **CallableStatement** → Executes stored procedures

Each type is used based on the requirement of the application.

The Statement Object

The **Statement object** is used when a query needs to be executed immediately without prior compilation.

The SQL query is passed as a String to the Statement object, which sends it to the DBMS for processing. The result returned by the database is stored in a **ResultSet object**.

executeQuery() Method

The executeQuery() method is used when the query retrieves data from the database (SELECT statements).

- Accepts SQL query as input
- Returns a **ResultSet object**
- The ResultSet contains rows, columns, and metadata

```
String query = "SELECT * FROM Customers";
Statement DataRequest = Db.createStatement();
ResultSet Results = DataRequest.executeQuery(query);
```

executeUpdate() Method

The executeUpdate() method is used when the query modifies the database.

It is used for:

- INSERT
- UPDATE
- DELETE
- DDL statements

This method does not return data. Instead, it returns an **integer value** indicating the number of rows affected.

```
String query = "UPDATE Customers SET PAID='Y' WHERE BALANCE='0'";
int rowsUpdated = DataRequest.executeUpdate(query);
```

The returned value can be used to confirm how many records were updated.

execute() Method

The execute() method is used when a query may return multiple results.

It is more general in nature and can handle both query results and update counts.

Closing the Statement

After executing the query and processing the results, the Statement object must be closed using the close() method.

SBC Karkala

- Frees system resources
- Closes all associated ResultSet objects
- Prevents resource locking issues

Failure to close the Statement may cause resources to remain occupied until garbage collection occurs.

Exception Handling

The methods of the Statement object, especially executeQuery() and executeUpdate(), may throw an **SQLException**.

This happens when:

- SQL syntax is incorrect
- DBMS cannot process the query

The error message generated by the DBMS is passed back to the application through this exception.

PreparedStatement Object

An SQL query must be compiled before the DBMS processes it. Normally, this compilation happens every time a query is executed using a Statement object. This is acceptable when the query is executed only once. However, if the same query is executed multiple times, repeated compilation becomes an overhead and reduces performance.

To overcome this, the **PreparedStatement object** is used. It allows a query to be **precompiled once** and executed multiple times with different values.

In a PreparedStatement, the query is written using **question mark (?) placeholders** instead of actual values. These placeholders are later replaced with actual values during execution.

For example:

String query = "SELECT * FROM Customers WHERE CustNumber = ?";

PreparedStatement pstatement = Db.preparedStatement(query);

The preparedStatement() method of the Connection object is used to create a PreparedStatement. This method also compiles the query.

Setting Values using setXXX()

The values for placeholders are assigned using **setXXX() methods**.

- The first parameter → position of the placeholder
- The second parameter → value to be inserted

Example:

pstatement.setString(1, "123");

Here, 1 indicates the first placeholder, and "123" is the value assigned.

Different setXXX() methods are available based on the data type of the value.

SBC Karkala

Executing PreparedStatement

After setting values, the query is executed using executeQuery().

ResultSet Results = pstatement.executeQuery();

The query does not need to be passed again because it is already associated with the PreparedStatement object.

Advantage of PreparedStatement

- Query is compiled only once
- Values can be changed without recompiling
- Improves performance when executed multiple times

The DBMS performs compilation and parameter binding. This process is called **late binding**, where values are associated with the query at execution time.

CallableStatement

The **CallableStatement object** is used to execute **stored procedures** from a J2ME application.

A stored procedure is a predefined block of code stored in the database. It is identified by a name and can be written in languages such as PL/SQL, TransactSQL, or others depending on the DBMS.

Types of Parameters

CallableStatement uses three types of parameters:

- **IN parameter** → used to pass values to the stored procedure
- **OUT parameter** → used to receive values from the stored procedure
- **INOUT parameter** → used for both input and output

Creating CallableStatement

The stored procedure is called using a query with placeholders:

String query = "{ CALL LastOrderNumber (?)}";

CallableStatement cstatement = Db.prepareCall(query);

The prepareCall() method of the Connection object is used to create the CallableStatement.

Registering OUT Parameter

Before executing the stored procedure, OUT parameters must be registered using registerOutParameter().

cstatement.registerOutParameter(1, Types.VARCHAR);

- First parameter → position of the parameter
- Second parameter → data type

Executing Stored Procedure

The stored procedure is executed using:

cstatement.execute();

SBC Karkala

Retrieving Output

After execution, the value of the OUT parameter is retrieved using getXXX() methods.

String lastOrderNumber = cstatement.getString(1);

Closing CallableStatement

The CallableStatement object must be closed after use:

cstatement.close();

ResultSet

A query is used to retrieve, update, or delete data in a database. When a query is executed using the executeQuery() method, it returns a **ResultSet object** that contains the data requested by the query.

The ResultSet object provides methods that allow a J2ME application to access and manipulate this data.

Structure of ResultSet

Data in a ResultSet is organized in the form of a **virtual table**:

- Rows → represent records
- Columns → represent fields

In addition to data, the ResultSet also contains **metadata**, such as:

- Column names
- Column size
- Data types

Virtual Cursor

The ResultSet uses a **virtual cursor** to point to rows in the table.

- Initially, the cursor is positioned **before the first row**
- To access data, the cursor must be moved to the first row

This is done using the next() method.

next() Method

The next() method:

- Moves the cursor to the next row
- Returns:
  - true → if data exists
  - false → if no more rows are available

This method is essential for traversing the ResultSet.

Retrieving Data using getXXX()

Once the cursor is positioned on a row, data is retrieved using **getXXX() methods**.

- The method depends on the data type

SBC Karkala

- Example: getString() for String values
- It requires one parameter:
  - Column number (starting from 1)

Example:
String name = Results.getString(1);
The data type of the method must match the data type of the column.

Column Order in ResultSet
Columns in the ResultSet appear in the same order as specified in the SELECT query.
Example:
SELECT CustomerFirstName, CustomerLastName FROM Customer

- Column 1 → First Name
- Column 2 → Last Name

So:

- getString(1) → First Name
- getString(2) → Last Name

Reading the ResultSet
To read data from a ResultSet:

1. Execute the query
2. Move cursor using next()
3. Retrieve values using getXXX()
4. Repeat until no more rows

Working Logic

- First call to next() checks if data exists
- If no data → program ends
- If data exists → enter loop
- Retrieve column values
- Move to next row and repeat

Example Flow

```
boolean Records = Results.next();
if (!Records) {
   System.out.println("No data returned");
}
else {
   do {
      FirstName = Results.getString(1);
      LastName = Results.getString(2);
      System.out.println(FirstName + " " + LastName);
   } while (Results.next());
}
```

SBC Karkala

Closing the Statement

After processing the ResultSet, the associated Statement object should be closed. This releases resources and ensures proper program execution.

Scrollable ResultSet

Earlier versions of JDBC allowed the virtual cursor to move only in the forward direction within a ResultSet. With the introduction of the JDBC 2.1 API, the cursor can now move both forward and backward, and can also be positioned at a specific row.

This enhanced capability is known as a **Scrollable ResultSet**.

Cursor Movement Methods

In addition to the next() method, several methods are available to control the position of the virtual cursor:

- first() → Moves cursor to the first row
- last() → Moves cursor to the last row
- previous() → Moves cursor to the previous row
- absolute(int row) → Moves cursor to the specified row number
- relative(int rows) → Moves cursor forward or backward relative to current position
- getRow() → Returns the current row number

The relative() method uses:

- Positive value → move forward
- Negative value → move backward

Types of Scrollable ResultSet

To use a scrollable ResultSet, the Statement object must be created with specific constants:

- **TYPE_FORWARD_ONLY**
  → Cursor moves only forward (default)
- **TYPE_SCROLL_INSENSITIVE**
  → Cursor moves in both directions
  → Does not reflect changes made by other applications
- **TYPE_SCROLL_SENSITIVE**
  → Cursor moves in both directions
  → Reflects changes made by other applications

Creating Scrollable ResultSet

The Statement object must be created with the required type:

DataRequest = Db.createStatement(TYPE_SCROLL_INSENSITIVE);

This allows the use of cursor movement methods.

SBC Karkala

Working of Cursor Movement

The cursor can be moved in different ways:

- Move to first row → first()
- Move to last row → last()
- Move one step back → previous()
- Move to a specific row → absolute(10)
- Move relative positions → relative(-2) or relative(2)

If the cursor moves before the first row or after the last row, no error is thrown.

Driver Support for Scrollable ResultSet

Not all JDBC drivers support scrollable ResultSet features. Therefore, it is necessary to check driver capability using DatabaseMetaData.

forward = meta.supportsResultSetType(ResultSet.TYPE_FORWARD_ONLY);
insensitive =
meta.supportsResultSetType(ResultSet.TYPE_SCROLL_INSENSITIVE);
sensitive = meta.supportsResultSetType(ResultSet.TYPE_SCROLL_SENSITIVE);

This helps determine whether scrolling is supported.

Specifying Number of Rows (Fetch Size)

Sometimes, all rows are not retrieved at once. The driver may fetch rows in batches. The number of rows fetched at a time can be controlled using:

DataRequest.setFetchSize(500);

- This sets the number of rows to be fetched from the DBMS at a time
- It is mainly used for performance tuning
- Not all drivers support this feature

Example: Scrollable ResultSet

```
String url = "jdbc:odbc:CustomerInformation";
String userID = "jim";
String password = "keogh";
Connection Db;
Statement DataRequest;
ResultSet Results;
try {
   Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
   Db = DriverManager.getConnection(url, userID, password);
   String query = "SELECT FirstName, LastName FROM Customers";
   // Creating scrollable ResultSet
   DataRequest = Db.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
                        ResultSet.CONCUR_READ_ONLY);

   Results = DataRequest.executeQuery(query);
```

SBCKarkala

```
  // Move to first row
   Results.first();
   System.out.println(Results.getString(1));

   // Move to last row
   Results.last();
   System.out.println(Results.getString(1));

   // Move to previous row
   Results.previous();
   System.out.println(Results.getString(1));

   // Move to specific row
   Results.absolute(2);
   System.out.println(Results.getString(1));

   // Move relative (forward/backward)
   Results.relative(1);
   System.out.println(Results.getString(1));

   DataRequest.close();
   Db.close();

} catch (Exception e) {
   System.out.println(e);
}
```

What this Example Shows

- first() → goes to first record
- last() → goes to last record
- previous() → moves one row back
- absolute(2) → jumps to row 2
- relative(1) → moves one row forward

Updatable ResultSet

A ResultSet can be made updatable so that rows within it can be modified similar to rows in a database table. This is achieved by creating the Statement object with the constant:

- CONCUR_UPDATABLE → allows updates
- CONCUR_READ_ONLY → prevents updates

```
DataRequest = Db.createStatement(ResultSet.CONCUR_UPDATABLE);
```

This enables the ResultSet to perform update, delete, and insert operations.

SBC Karkala

Operations on Updatable ResultSet

There are three main operations that can be performed:

- Updating values in a row
- Deleting a row
- Inserting a new row

1. Updating a Value in ResultSet

To modify a value in the current row:

- Use updateXXX() method
- Then call updateRow() to apply changes

The updateXXX() method:

- First parameter → column number or name
- Second parameter → new value

Example:

Results.updateString("LastName", "Smith");

Results.updateRow();

To assign a NULL value: Results.updateNull(columnNumber);

Important point:

- The change occurs in the ResultSet
- The actual table is not updated unless an update query is executed

2. Deleting a Row in ResultSet

The deleteRow() method is used to remove a row.

- Cursor should be positioned on the row to be deleted
- The method deletes the current row

Example: Results.deleteRow(0);

This:

- Removes the row from ResultSet
- Also deletes it from the underlying database

3. Inserting a Row in ResultSet

A new row can be inserted using:

- updateXXX() → set column values
- insertRow() → insert the row

Example:

Results.updateString(1, "Tom");

Results.updateString(2, "Smith");

Results.insertRow();

Important points:

- Column order is based on SELECT query
- Multiple columns can be set before insertion
- The new row is added to both ResultSet and database

SBCKarkala

Working Summary

- Enable updatable ResultSet using CONCUR_UPDATABLE
- Move cursor to required row
- Perform operation:
  - updateXXX() + updateRow()
  - deleteRow()
  - updateXXX() + insertRow()

Example: Updatable ResultSet

Updating a Value

```
String url = "jdbc:odbc:CustomerInformation";
String userID = "jim";
String password = "keogh";
Connection Db;
Statement DataRequest;
ResultSet Results;
try {
   Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
   Db = DriverManager.getConnection(url, userID, password);
   String query = "SELECT FirstName, LastName FROM Customers WHERE
FirstName='Mary' AND LastName='Jones'";
   DataRequest = Db.createStatement(ResultSet.CONCUR_UPDATABLE);
   Results = DataRequest.executeQuery(query);

   if (Results.next()) {
      Results.updateString("LastName", "Smith");
      Results.updateRow();   // apply update
   }

   DataRequest.close();
   Db.close();
} catch (Exception e) {
   System.out.println(e);
}
```

What this Example Shows

- CONCUR_UPDATABLE → makes ResultSet editable
- updateString() → changes value
- updateRow() → confirms update

SBC Karkala

Insert Example

```
Results.updateString(1, "Tom");
Results.updateString(2, "Smith");
Results.insertRow();
```

Delete Example

```
Results.deleteRow(0);
```

Transaction Processing

A transaction in a database consists of a group of SQL statements that must all be executed successfully for the transaction to be completed. If any one of the statements fails, all previously executed statements must be **rolled back**.

This is similar to a real-world transaction where all steps must complete successfully; otherwise, the entire process is cancelled.

A transaction is completed only when the commit() method is called. Before this, all operations can be reversed using rollback().

Commit and Rollback

- commit() : Permanently saves all changes made in the transaction
- rollback(): → Reverses all changes made before commit

Once commit() is executed, changes cannot be undone

AutoCommit Mode

By default, the DBMS uses **AutoCommit = true**.

- Each SQL statement is automatically committed
- No need to call commit() explicitly

For transaction processing:

```
Database.setAutoCommit(false);
```

- Disables AutoCommit
- Allows multiple statements to be treated as a single transaction

After completion:

```
Database.setAutoCommit(true);
```

- Re-enables AutoCommit

Transaction Execution

Steps involved:

1. Disable AutoCommit
2. Execute multiple SQL statements
3. Call commit() if all succeed
4. If any error occurs → call rollback()

SBC Karkala

Example Flow

Database.setAutoCommit(false);
DataRequest.executeUpdate(query1);
DataRequest.executeUpdate(query2);
Database.commit();
If an exception occurs:
Database.rollback();

Savepoints

A transaction may contain multiple steps, and sometimes only part of the transaction needs to be rolled back.

A **savepoint** is a marker within a transaction that defines where rollback should stop.

- Created using setSavepoint()
- Identified by a unique name

Example:

Savepoint sp1 = Database.setSavepoint("sp1");
To rollback up to savepoint: Database.rollback(sp1);
To remove savepoint: Database.releaseSavepoint(sp1);

Batch Statements

Batch processing allows multiple SQL statements to be grouped and executed as a single transaction.

Working of Batch: batch sql statement

- Add SQL statements using addBatch()
- Execute all statements using executeBatch()
- Commit the transaction

Example:

DataRequest.addBatch(query1);
DataRequest.addBatch(query2);
int[] updated = DataRequest.executeBatch();
Database.commit();

Exception Handling in Batch

If an error occurs, a **BatchUpdateException** is thrown.

- Provides details of executed statements
- Allows error analysis and recovery

Keeping ResultSet Objects Open (Holdability)

Normally, when the commit() method is called, all associated ResultSet objects are automatically closed.

However, in some situations, a J2ME application may need to **keep the ResultSet open even after commit**. This behavior is called **holdability**.

SBC Karkala

Holdability Options

Holdability is controlled by passing constants to the createStatement() method:

- **HOLD_CURSORS_OVER_COMMIT**
  → Keeps ResultSet open after commit()
- **CLOSE_CURSORS_AT_COMMIT**
  → Closes ResultSet when commit() is called

RowSet

A **RowSet** is a JDBC object used to store rows of data similar to a ResultSet, but with additional flexibility.

A RowSet is mainly used in **disconnected environments**, meaning:

- It can hold data without maintaining a continuous connection to the DBMS
- Suitable for applications like J2ME where continuous connectivity may not be possible

Features of RowSet

- Contains rows of data from one or more tables
- Can operate without an active database connection
- Useful for distributed or mobile applications

RowSet Events

A RowSet generates events whenever:

- The cursor is moved
- Data in rows is modified

These events are represented by **RowSetEvent objects**.

When an event occurs:

- The RowSet creates a RowSetEvent
- Sends it to all registered listeners

RowSetListener

A **RowSetListener** is a class that listens for RowSet events.

It must implement the RowSetListener interface and define three methods:

public void cursorMoved(RowSetEvent event)

public void rowChanged(RowSetEvent event)

public void rowSetChanged(RowSetEvent event)

Methods Description

- cursorMoved(): Triggered when cursor position changes
- rowChanged(): Triggered when a row is modified
- rowSetChanged(): Triggered when the entire RowSet is modified

SBC Karkala

Registering RowSetListener

A listener must be created and registered with the RowSet:

MyRowSetListener rowsetlistener = new MyRowSetListener();

rowset.addRowSetListener(rowsetlistener);

To remove the listener: rowset.removeRowSetListener(rowsetlistener);

Autogenerated Keys

Many databases automatically generate unique keys (such as primary keys) when new rows are inserted.

The getGeneratedKeys() method of the Statement object is used to retrieve these keys.

Metadata

Metadata is **data about data**. It provides information about the structure and properties of the database and its components.

A J2ME application accesses metadata using the **DatabaseMetaData interface**. This interface provides information about: Databases, Tables , Columns , Indexes , Other DBMS-related details

Accessing Database Metadata

Metadata about the database is retrieved using the getMetaData() method of the **Connection object**.

DatabaseMetaData meta = Db.getMetaData();

This method returns a **DatabaseMetaData object**, which contains detailed information about the database.

Common DatabaseMetaData Methods

Some commonly used methods include:

- getDatabaseProductName(): Returns the database product name
- getUserName(): Returns the current user name
- getURL(): Returns the database URL
- getSchemas(): Returns available schema names
- getPrimaryKeys(): Returns primary key information
- getProcedures(): Returns stored procedure names
- getTables(): Returns table names in the database

ResultSet Metadata

Metadata can also describe the structure of the ResultSet returned by a query.

This type of metadata is accessed using the getMetaData() method of the **ResultSet object**.

ResultSetMetaData rm = Results.getMetaData();

This method returns a **ResultSetMetaData object**.

SBC Karkala

Common ResultSetMetaData Methods

- getColumnCount(): → Returns number of columns in the ResultSet
- getColumnName(int number): → Returns the name of the specified column
- getColumnType(int number): → Returns the data type of the specified column

---

Data Types

The setXXX() and getXXX() methods are used to set and retrieve values of specific data types.

The **XXX** in these method names is replaced with the name of the data type.

- setXXX() → used to set a value
- getXXX() → used to retrieve a value

The appropriate data type name must be used in place of XXX.

Some examples of database data types:

SQL type	Java type
CHAR	String
VARCHAR	String
INTEGER	Integer

Exceptions

JDBC methods throw three types of exceptions:

SQLException, SQLWarning, DataTruncation

1. SQLException

SQLException is the most commonly occurring exception.

- It indicates errors related to SQL operations
- Usually caused by:
  - SQL syntax errors
  - Database connectivity issues
  - Accessing closed objects

Methods associated with SQLException:

- getNextException(): → Returns details of the next exception
- getErrorCode(): → Returns vendor-specific error code

2. SQLWarning

SQLWarning represents warnings received from the DBMS.

- It does not stop program execution
- Indicates non-critical issues

Methods used:

- getWarnings(): → Retrieves the warning
- getNextWarning(): → Retrieves subsequent warnings

3. DataTruncation

DataTruncation occurs when:

- Data is lost due to truncation

This happens when the size of data exceeds the allowed limit.

SBC Karkala